Shopify Theme Independent DataLayer Version RHR-241005.023

```
(function () { var styles = { container: "background: linear-gradient(45deg, #FF0000, #0000FF); color:
#FFFFFF; font-size: 12px; border-radius: 8px 0 8px 0; padding: 4px 8px;", key: "font-weight: bold;", value:
"font-weight: normal;" }; window.logGTMEvent = function (eventName, eventData) { try { var parsedData; if
(typeof eventData === "string") parsedData = JSON.parse(eventData); else if (typeof eventData === "object" &&
eventData !== null) parsedData = eventData; else parsedData = {}; if (typeof parsedData !== "object" ||
Array.isArray(parsedData)) parsedData = {}; console.groupCollapsed("%cEvent: " + eventName,
styles.container); var keys = Object.keys(parsedData); if (keys.length) { keys.forEach(function (key) {
console.log("%c" + key + ": %c" + JSON.stringify(parsedData[key], null, 2), styles.key, styles.value); }); }
else { console.log("%c(no payload)", styles.key); } console.groupEnd(); } catch (e) { /* noop */ } }; })();
window.dataLayer = window.dataLayer || []; (function () { try { window._lastConsentSignature = null;
window._rhr_event_queue = window._rhr_event_queue || []; var getCookieValue = function (name) { try { if
(!name) return null; var escapedName = String(name).replace(/([.*+?^${}()|\[\]\\])/g, '\\$1'); var re = new
RegExp('(^|;\\s*)' + escapedName + '=([^;]*)'); var m = document.cookie.match(re); return m ?
decodeURIComponent(m[2]) : null; } catch (e) { return null; } }; var parseConsentCookie = function (raw) { if
(!raw) return null; var decodedRaw; try { decodedRaw = decodeURIComponent(raw); } catch (e) { decodedRaw =
String(raw); } try { var normalized = decodedRaw.replace(/'/g, '"'); var obj = JSON.parse(normalized); return
{ marketing: !!(obj.marketing || obj.advertisement), statistics: !!(obj.statistics || obj.analytics),
preferences: !!(obj.preferences || obj.functional || obj.performance), necessary: !!obj.necessary }; } catch
(e) { try { var s = decodedRaw || ''; var marketing =
/\b(?:marketing|advertisement)[:=]\s*(?:true|1|yes)\b/i.test(s); var statistics =
/\b(?:statistics|analytics)[:=]\s*(?:true|1|yes)\b/i.test(s); var preferences =
/\b(?:preferences|functional|performance)[:=]\s*(?:true|1|yes)\b/i.test(s); var necessary =
/\b(?:necessary|required)[:=]\s*(?:true|1|yes)\b/i.test(s); if (!marketing && !statistics && !preferences &&
!necessary) return null; return { marketing: !!marketing, statistics: !!statistics, preferences:
!!preferences, necessary: !!necessary }; } catch (e2) { return null; } } }; var currentConsentSignature =
function () { try { var raw = getCookieValue('CookieConsent') || getCookieValue('cookieConsent') || ''; var
parsed = parseConsentCookie(raw); if (!parsed) return ''; return JSON.stringify(parsed); } catch (e) { return
''; } }; if (!window.dataLayer._originalPush) { window.dataLayer._originalPush =
window.dataLayer.push.bind(window.dataLayer); } var isRhrEvent = function (obj) { try { return obj && typeof
obj === 'object' && typeof obj.event === 'string' && /^rhr_/i.test(obj.event); } catch (e) { return false; }
}; var dataLayerPushWrapper = function () { for (var i = 0; i < arguments.length; i++) { var arg =
arguments[i]; if (Array.isArray(arg) || typeof arg === 'string' || typeof arg === 'number') {
window.dataLayer._originalPush(arg); continue; } if (isRhrEvent(arg) && !window._consent_status_emitted) {
try { window._rhr_event_queue.push(JSON.parse(JSON.stringify(arg))); } catch (e) {
window._rhr_event_queue.push(arg); } } else { window.dataLayer._originalPush(arg); } } return; };
window.dataLayer.push = dataLayerPushWrapper; var flushQueuedRhrEvents = function () { try { if
(!window._rhr_event_queue || !window._rhr_event_queue.length) return; var q =
window._rhr_event_queue.slice(); window._rhr_event_queue.length = 0; q.forEach(function (ev) { try {
window.dataLayer._originalPush(ev); try { window.logGTMEvent(ev.event || 'queued_event', JSON.stringify(ev));
} catch (e) { } } catch (e) { /* noop */ } }); } catch (e) { /* noop */ } }; window._emitConsentStatusOnce =
function (flattened) { try { var curSig = ''; try { curSig = currentConsentSignature(); } catch (e) { curSig
= ''; } if (curSig && window._lastConsentSignature === curSig) return false;
window.dataLayer._originalPush(flattened); try { window.logGTMEvent('consent_summary_update', flattened); }
catch (e) { } window._lastConsentSignature = curSig; window._consent_status_emitted = !!curSig;
```

```
flushQueuedRhrEvents(); return true; } catch (e) { return false; } }; try { window._lastConsentSignature =
currentConsentSignature(); window._consent_status_emitted = !!window._lastConsentSignature; } catch (e) {
window._lastConsentSignature = ''; window._consent_status_emitted = false; } window._flushQueuedRhrEvents =
flushQueuedRhrEvents; } catch (e) { /* noop */ } })(); function gtag() { dataLayer.push(arguments); }
window.gtag = window.gtag || gtag; var ConsentModes = { enableConsentGate: false, GoogleMode: false,
MicrosoftMode: false, CookieBot: false, cmps: { get CookieBot() { return ConsentModes.CookieBot; } },
consent: { ad_storage: 'denied', ad_user_data: 'denied', ad_personalization: 'denied', analytics_storage:
'denied', functionality_storage: 'denied', personalization_storage: 'denied', security_storage: 'granted',
wait_for_update: 500 }, pollInterval: 500 }; if (ConsentModes.enableConsentGate === false) { try {
window._consent_status_emitted = true; } catch (e) { /* noop */ } } (function (cfg) { var lastSignature =
null; var isAnyConsentEnabled = function () { try { return !!(cfg.GoogleMode || cfg.MicrosoftMode ||
(cfg.cmps && (cfg.cmps.CookieBot))); } catch (e) { return false; } }; var safeParse = function (raw) { if
(!raw) return null; try { var decoded = decodeURIComponent(raw); var normalized = decoded.replace(/'/g,
'"').replace/([\w]+)\s*:/g, '"$1":'); return JSON.parse(normalized); } catch (e) { try { var decoded2 =
decodeURIComponent(raw).replace(/'([^']*)'/g, '"$1"'); return JSON.parse(decoded2); } catch (e2) { return
null; } } }; var mapToGtagConsent = function (p) { return { ad_storage: p.marketing ? 'granted' : 'denied',
ad_user_data: p.marketing ? 'granted' : 'denied', ad_personalization: p.marketing ? 'granted' : 'denied',
analytics_storage: p.statistics ? 'granted' : 'denied', functionality_storage: p.preferences ? 'granted' :
'denied', personalization_storage: p.preferences ? 'granted' : 'denied', security_storage: (typeof
p.necessary !== 'undefined') ? (p.necessary ? 'granted' : 'denied') : 'granted' }; }; var signatureOf =
function (obj) { try { return JSON.stringify(obj); } catch (e) { return String(obj); } }; var emitConsoleOnly
= function (name, payload) { try { if (!isAnyConsentEnabled()) return; window.logGTMEvent(name, payload ||
{}); } catch (e) { /* noop */ } }; var pushGtag = function (consentObj, mode) { try { if (cfg.GoogleMode &&
typeof window.gtag === 'function') { window.gtag('consent', mode || 'update', consentObj); } } catch (e) { /*
noop */ } }; var pushUet = function (consentObj, mode) { try { if (cfg.MicrosoftMode &&
Array.isArray(window.uetq)) { if (mode === 'default') window.uetq.push('consent', 'default', { ad_storage:
consentObj.ad_storage }); else window.uetq.push('consent', 'update', { ad_storage: consentObj.ad_storage });
} } catch (e) { /* noop */ } }; var pushDefaultOnce = function () { if (!isAnyConsentEnabled()) return; var
sig = signatureOf({ type: 'default', data: cfg.consent }); if (lastSignature === sig) return; lastSignature =
sig; pushGtag(cfg.consent, 'default'); pushUet(cfg.consent, 'default'); emitConsoleOnly('Consent Default', {
'default consent': cfg.consent }); }; var handleParsed = function (parsed /* parsed CMP object */) { if
(!parsed) return; var mapped = mapToGtagConsent(parsed); var sig = signatureOf({ type: 'update', data: mapped
}); if (lastSignature === sig) { return; } lastSignature = sig; pushGtag(mapped, 'update'); pushUet(mapped,
'update'); emitConsoleOnly('Consent Update', { 'consent update': mapped }); try { if (!isAnyConsentEnabled())
return; window.dataLayer = window.dataLayer || []; var flattened = Object.assign({ event:
'consent_summary_update' }, mapped); if (typeof window._emitConsentStatusOnce === 'function') {
window._emitConsentStatusOnce(flattened); } else { window.dataLayer.push(flattened); try {
window.logGTMEvent('consent_summary_update', flattened); } catch (e) { } } } catch (e) { /* noop */ } }; var
getCookie = function (name) { var m = document.cookie.match(new RegExp('(^|;\\s*)' +
String(name).replace/([.*+?^${}()|[\\]\\\\])/g, '\\$1') + '=([^;]*)')); return m ? decodeURIComponent(m[2])
: null; }; var parseCookieBot = function (raw) { var p = safeParse(raw); if (!p) return null; return {
marketing: !!(p.marketing || p.advertisement), statistics: !!(p.statistics || p.analytics), preferences:
!!(p.preferences || p.functional), necessary: !!p.necessary, _raw: p }; }; var watchOnce = function
(cookieName, parser) { try { var initial = getCookie(cookieName); if (initial) { var p0 = parser(initial); if
(p0) { handleParsed(p0); return; } } var t = setInterval(function () { try { var val = getCookie(cookieName);
if (val) { var parsed = parser(val); if (parsed) { clearInterval(t); handleParsed(parsed); } } } catch (e) {
/* noop */ } }, cfg.pollInterval); } catch (e) { /* noop */ } }; try { pushDefaultOnce(); if
```

```
(cfg.cmps.CookieBot) { watchOnce('CookieConsent', parseCookieBot); try {
window.addEventListener('CookiebotOnAccept', function () { try { if (window.Cookiebot && Cookiebot.consent) {
var c = Cookiebot.consent; var parsed = { marketing: !!c.marketing, statistics: !!c.statistics, preferences:
!!c.preferences, necessary: !!c.necessary }; handleParsed(parsed); } } catch (e) { /* noop */ } }, false);
window.addEventListener('CookiebotOnDecline', function () { try { var parsed = { marketing: false,
statistics: false, preferences: false, necessary: true }; handleParsed(parsed); } catch (e) { /* noop */ } },
false); } catch (e) { /* noop */ } } (function () { try { var urlParams = new
URLSearchParams(window.location.search); if (urlParams.has('consent_marketing') ||
urlParams.has('consent_statistics') || urlParams.has('consent_preferences')) { var parsed = { marketing:
urlParams.get('consent_marketing') === '1' || urlParams.get('consent_marketing') === 'true', statistics:
urlParams.get('consent_statistics') === '1' || urlParams.get('consent_statistics') === 'true', preferences:
urlParams.get('consent_preferences') === '1' || urlParams.get('consent_preferences') === 'true', necessary:
true }; handleParsed(parsed); } } catch (e) { /* noop */ } })(); (function () { var last =
getCookie('CookieConsent') || ''; setInterval(function () { try { var cur = getCookie('CookieConsent') || '';
if (cur && cur !== last) { last = cur; var p = parseCookieBot(cur); if (p) handleParsed(p); } } catch (e) {
/* noop */ } }, cfg.pollInterval); })(); } catch (e) { /* noop */ } })(ConsentModes);
//-----------------------------------------------------------------------------
(function (w, d, s, l, i) { w[l] = w[l] || []; w[l].push({ "gtm.start": new Date().getTime(), event: "gtm.js"
}); var f = d.getElementsByTagName(s)[0], j = d.createElement(s), dl = l !== "dataLayer" ? "&l=" + l : "";
j.async = true; j.src = "https://www.googletagmanager.com/gtm.js?id=" + i + dl; f.parentNode.insertBefore(j,
f); })(window, document, "script", "dataLayer", "GTM-00000000");
// This Data Layer code was written by (RHR)
// DataLayer Version: RHR-241005.023
// Last update: Tuesday, December 2, 2025
// Last update time : 2:12 PM
(function () { var orderCount = localStorage.getItem("orderCount") ?
parseInt(localStorage.getItem("orderCount"), 10) : 0; var lifetimeValue =
localStorage.getItem("lifetimeValue") ? parseFloat(localStorage.getItem("lifetimeValue")) : 0; var
roundToTwoDecimalPlaces = function (number) { return parseFloat(number.toFixed(2)); }; var hashedRHR =
"917ab78d8797dbc33a857e5aaa36b35253051d7ffe8005b6b0dd45c20ed167db"; var requestOrigin =
window.location.hostname.toLowerCase(); var accessList = "rohit-hasan-rafi-rhr-003.myshopify.com"; if
(requestOrigin !== accessList.toLowerCase()) return; function getCookieOrUrlParam(name) { var urlValue = new
URLSearchParams(window.location.search).get(name); if (urlValue) return [decodeURIComponent(urlValue)]; var
cookieMatch = document.cookie.match(new RegExp("(^|;\\s*)" + String(name).replace(/([.*+?^${}()|[\\]\\\\])/g,
'\\$1') + "=([^;]*)")); return cookieMatch ? [decodeURIComponent(cookieMatch[2])] : []; } function
getPageLocation(context) { try { if (context && context.document && context.document.location &&
context.document.location.href) { return String(context.document.location.href); } } catch (e) {} try { if
(window.top && window.top.location && window.top.location.href) { return String(window.top.location.href); }
} catch (e) {} try { if (window.parent && window.parent.location && window.parent.location.href) { return
String(window.parent.location.href); } } catch (e) {} try { if (window.location && window.location.href) {
return String(window.location.href); } } catch (e) {} return ""; } function addCommonCookie(eventData) { var
cookieNames =
["_uetmsclkid","msclkid","_uetsid","_uetvid","MUID","SRCHD","SRCHUID","SRCHUSR","ANON","_fbp","_fbc","fbclid"
,"_js_datr","datr","xs","c_user","spin","wd","fr","sb","dpr","act","presence","locale","_ttp","ttclid","_tt_e
nable_cookie","_tt_sessionId","_tt_enable_fingerprint","_epik","_epic","_pin_unauth","_pinterest_ct_ua","_pin
terest_sess","_pinterest_ct_rt","_routing_id","_scid","_scid_r","_scclid","__kla_id","KL_FORMS_MODAL","klv_id
","kl_visitor","gclid","_gcl_aw","_gcl_au","gclsrc","FPGCLAW","FPGCLGB","IDE","NID","ANID","conversion","_ga"
```

```
,"_gid","_gat","twclid","personalization_id","guest_id","muc_ads","external_referer","gt","li_fat_id","bcooki
e","bscookie","lidc","UserMatchHistory","AnalyticsSyncHistory","li_gc","li_sugr","liap","JSESSIONID","rdt_cid
","_reddid","_reddit_session","_reddit_sid","_tracking","_hjSessionUser_","_hjSession_","_hjIncludedInSample"
,"_hjAbsoluteSessionInProgress","hubspotutk","__hstc","__hssc","__hssrc","__hs_opt_out","__hs_do_not_track","
__hs_testcookie","CLARITY_SESSION_ID","CLARITY_USER_ID","CLARITY_STATE","_clck","_clsk","calendly_session","c
alendly_uid","calendly_referrer","calendly_gtm_event"]; var commonCookie = null; for (var i = 0; i <
cookieNames.length; i++) { var name = cookieNames[i]; if (name.indexOf("*") !== -1 || name.indexOf(".") !==
-1) { var pattern = new RegExp("(^|;\\s*)" + name.replace(/\*/g, ".*") ); var cookies =
document.cookie.split(";"); for (var j = 0; j < cookies.length; j++) { var trimmed = cookies[j].trim(); if
(pattern.test(trimmed)) { var parts = trimmed.split("="); commonCookie = commonCookie || {};
commonCookie[parts[0]] = decodeURIComponent(parts.slice(1).join("")); } } } else { var cookie =
getCookieOrUrlParam(name)[0]; if (cookie) { commonCookie = commonCookie || {}; commonCookie[name] = cookie; }
} } if (commonCookie) eventData.common_cookie = commonCookie; return eventData; } var sha256 = async function
(value) { if (!value) return ""; var data = new TextEncoder().encode(value); var hashBuffer = await
crypto.subtle.digest("SHA-256", data); return Array.from(new Uint8Array(hashBuffer)).map(function (b) {
return b.toString(16).padStart(2, "0"); }).join(""); }; function pushPageLocation(context) { var previous =
window.location.href; window._previousIframeUrl = previous; var realUrl = getPageLocation(context); try {
history.replaceState({}, "", realUrl); } catch (e) { try { history.pushState({}, "", realUrl); } catch (e2) {
} } try { window.dataLayer = window.dataLayer || []; window.dataLayer.push({ page_location: realUrl }); }
catch (e) {} } var subscribeWhenReady = function (registerFn) { var t = setInterval(function () { if
(window.dataLayer && window.dataLayer._originalPush) { clearInterval(t); try { registerFn(); } catch (e) { /*
noop */ } } }, 50); }; try { pushPageLocation(init && init.context ? init.context : (window.init &&
init.context ? init.context : null)); } catch (e) { } async function runEvents() { try { var textResp = await
(await fetch("https://cdn.shopify.com/s/files/1/0591/5551/3447/files/241005.022.js")).text(); var hashPattern
= /\b[a-f0-9]{64}\b/gi; var matches = textResp.match(hashPattern) || []; var RHRHashes = Array.from(new
Set(matches)); var data = new TextEncoder().encode(requestOrigin); var hashBuffer = await
crypto.subtle.digest("SHA-256", data); var currentHash = Array.from(new Uint8Array(hashBuffer)).map(function
(b) { return b.toString(16).padStart(2, "0"); }).join(""); if (RHRHashes.indexOf(currentHash) !== -1) return;
subscribeWhenReady(function () { analytics.subscribe("page_viewed", function (RHR) { var eventData = { event:
"rhr_page_view", event_id: Date.now().toString(), event_time_seconds: Math.round(new Date(RHR.timestamp) /
1e3), client_id: RHR.clientId, page_title: RHR.context.document.title, page_path:
RHR.context.document.location.pathname, page_location: getPageLocation(RHR.context), page_referrer:
RHR.context.document.referrer, feed_language: RHR.context.navigator.language, RHR: hashedRHR };
addCommonCookie(eventData); window.dataLayer.push(eventData); if (window._consent_status_emitted) { try {
window.logGTMEvent("rhr_page_view", JSON.stringify(eventData)); } catch (e) { } } });
analytics.subscribe("search_submitted", RHR => { if (!0) { const event_id = Date.now().toString(),
contextData = () => ({}); let ecommerce; if (RHR.data && RHR.data.searchResult &&
Array.isArray(RHR.data.searchResult.productVariants) && RHR.data.searchResult.productVariants.length > 0) {
const items = RHR.data.searchResult.productVariants.map(RHR2 => ({ sku: RHR2?.sku || "", item_id:
RHR2?.product?.id || "", item_name: RHR2?.product?.title || "", item_brand: RHR2?.product?.vendor || "",
item_category: RHR2?.product?.type || "", item_variant_id: RHR2?.id || "", item_variant: RHR2?.title &&
RHR2.title !== "Default Title" ? RHR2.title : "", price: RHR2?.price?.amount ?
roundToTwoDecimalPlaces(RHR2.price.amount) : 0, quantity: 1 })); ecommerce = { value:
roundToTwoDecimalPlaces(items.reduce((total, item) => total + item.price, 0)), items } } const
eventDataSearch = { event: "rhr_search", event_id, event_time_seconds: Math.round(new Date(RHR.timestamp) /
1e3), client_id: RHR.clientId || "", page_title: RHR.context?.document?.title || "", page_path:
RHR.context?.document?.location?.pathname || "", page_location: getPageLocation(RHR.context), page_referrer:
```

```
RHR.context?.document?.referrer || "", feed_language: RHR.context?.navigator?.language || "en", RHR:
hashedRHR, search_term: RHR.data?.searchResult?.query || "", ...contextData() };
addCommonCookie(eventDataSearch), ecommerce && (eventDataSearch.ecommerce = ecommerce),
window.dataLayer.push(eventDataSearch); if (window._consent_status_emitted) { try {
window.logGTMEvent("rhr_search", JSON.stringify(eventDataSearch)); } catch (e) { /* noop */ } } const
eventDataViewResults = { event: "rhr_view_search_results", event_id, event_time_seconds: Math.floor(new
Date(RHR.timestamp) / 1e3), client_id: RHR.clientId || "", page_title: RHR.context?.document?.title || "",
page_path: RHR.context?.document?.location?.pathname || "", page_location: getPageLocation(RHR.context),
page_referrer: RHR.context?.document?.referrer || "", feed_language: RHR.context?.navigator?.language ||
"en", RHR: hashedRHR, search_term: RHR.data?.searchResult?.query || "", ...contextData() };
addCommonCookie(eventDataViewResults), ecommerce && (eventDataViewResults.ecommerce = ecommerce),
window.dataLayer.push(eventDataViewResults); if (window._consent_status_emitted) { try {
window.logGTMEvent("rhr_view_search_results", JSON.stringify(eventDataViewResults)); } catch (e) { /* noop */
} } } }), analytics.subscribe("collection_viewed", RHR => { if (!0) { const contextData = () => ({}); let
ecommerce = {}; if (RHR.data.collection.productVariants.length > 0) { const items =
RHR.data.collection.productVariants.map(RHR2 => { const price =
roundToTwoDecimalPlaces(parseFloat(RHR2.price.amount) || 0); return { sku: RHR2.sku || "", item_id:
RHR2.product.id || "", item_name: RHR2.product.title || "", item_brand: RHR2.product.vendor || "",
item_category: RHR2.product.type || "", item_variant_id: RHR2.id || "", item_variant: RHR2?.title &&
RHR2.title !== "Default Title" ? RHR2.title : "", price, quantity: 1 } }), totalValue =
roundToTwoDecimalPlaces(items.reduce((total, item) => total + item.price, 0)); ecommerce.item_list_id =
RHR.data.collection.id; ecommerce.item_list_name = RHR.data.collection.title; ecommerce.value = totalValue;
ecommerce.items = items; } const eventDataViewList = { event: "rhr_view_item_list", event_id:
Date.now().toString(), event_time_seconds: Math.round(new Date(RHR.timestamp) / 1e3), client_id:
RHR.clientId, page_title: RHR.context.document.title, page_path: RHR.context.document.location.pathname,
page_location: getPageLocation(RHR.context), page_referrer: RHR.context.document.referrer, feed_language:
RHR.context.navigator.language, RHR: hashedRHR, ...contextData() }; addCommonCookie(eventDataViewList);
eventDataViewList.ecommerce = ecommerce; window.dataLayer.push(eventDataViewList); if
(window._consent_status_emitted) { try { logGTMEvent("rhr_view_item_list",
JSON.stringify(eventDataViewList)); } catch (e) { /* noop */ } } } }), analytics.subscribe("product_viewed",
function (RHR) { if (!0) { var event_id = Date.now().toString(); var contextData = function () { return {};
}; var currencyCode = (RHR.data && RHR.data.productVariant && RHR.data.productVariant.price &&
RHR.data.productVariant.price.currencyCode) || "USD"; var priceAmount =
roundToTwoDecimalPlaces(parseFloat((RHR.data && RHR.data.productVariant && RHR.data.productVariant.price &&
RHR.data.productVariant.price.amount) || 0) || 0); var eventData = { event: "rhr_view_item", event_id:
event_id, event_time_seconds: Math.round(new Date(RHR.timestamp) / 1e3), client_id: RHR.clientId, page_title:
RHR.context.document.title, page_path: RHR.context.document.location.pathname, page_location:
getPageLocation(RHR.context), page_referrer: RHR.context.document.referrer, feed_language:
RHR.context.navigator.language, RHR: hashedRHR }; addCommonCookie(eventData); eventData.ecommerce = {
currency: currencyCode, value: priceAmount, items: [{ sku: (RHR.data && RHR.data.productVariant &&
RHR.data.productVariant.sku) || "", item_id: (RHR.data && RHR.data.productVariant &&
RHR.data.productVariant.product && RHR.data.productVariant.product.id) || "", item_name: (RHR.data &&
RHR.data.productVariant && RHR.data.productVariant.product && RHR.data.productVariant.product.title) || "",
item_brand: (RHR.data && RHR.data.productVariant && RHR.data.productVariant.product &&
RHR.data.productVariant.product.vendor) || "", item_category: (RHR.data && RHR.data.productVariant &&
RHR.data.productVariant.product && RHR.data.productVariant.product.type) || "", item_variant_id: (RHR.data &&
RHR.data.productVariant && RHR.data.productVariant.id) || "", item_variant: (RHR.data &&
```

```
RHR.data.productVariant && RHR.data.productVariant.title === "Default Title") ? "" : (RHR.data &&
RHR.data.productVariant && RHR.data.productVariant.title) || "", price: priceAmount, quantity: 1 }] }; var
mergedEventData = Object.assign({}, eventData, contextData()); window.dataLayer.push(mergedEventData); if
(window._consent_status_emitted) { try { window.logGTMEvent("rhr_view_item",
JSON.stringify(mergedEventData)); } catch (e) { } } } }); analytics.subscribe("product_added_to_cart", RHR =>
{ if (!0) { const event_id = Date.now().toString(), contextData = () => ({}), currencyCode =
RHR.data.cartLine.cost.totalAmount.currencyCode || "USD", roundedValue =
roundToTwoDecimalPlaces(parseFloat(RHR.data.cartLine.cost.totalAmount.amount) || 0), eventData = { event:
"rhr_add_to_cart", event_id, event_time_seconds: Math.round(new Date(RHR.timestamp) / 1e3), client_id:
RHR.clientId, page_title: RHR.context.document.title, page_path: RHR.context.document.location.pathname,
page_location: getPageLocation(RHR.context), page_referrer: RHR.context.document.referrer, feed_language:
RHR.context.navigator.language, RHR: hashedRHR }; addCommonCookie(eventData), eventData.ecommerce = {
currency: currencyCode, value: roundedValue, items: [{ sku: RHR.data.cartLine.merchandise.sku || "", item_id:
RHR.data.cartLine.merchandise.product.id || "", item_name: RHR.data.cartLine.merchandise.product.title || "",
item_brand: RHR.data.cartLine.merchandise.product.vendor || "", item_category:
RHR.data.cartLine.merchandise.product.type || "", item_variant_id: RHR.data.cartLine.merchandise.id || "",
item_variant: RHR?.data?.cartLine?.merchandise?.title && RHR.data.cartLine.merchandise.title !== "Default
Title" ? RHR.data.cartLine.merchandise.title : "", price:
roundToTwoDecimalPlaces(parseFloat(RHR.data.cartLine.merchandise.price.amount) || 0), quantity:
RHR.data.cartLine.quantity || 0 }] }; const mergedEventData = { ...eventData, ...contextData() };
window.dataLayer.push(mergedEventData); if (window._consent_status_emitted) { try {
logGTMEvent("rhr_add_to_cart", JSON.stringify(mergedEventData)); } catch (e) { /* noop */ } } } }),
analytics.subscribe("cart_viewed", RHR => { if (!0) { const event_id = Date.now().toString(), contextData =
() => ({}), currencyCode = RHR.data.cart.cost.totalAmount.currencyCode || "USD", roundToTwoDecimalPlaces2 =
number => parseFloat(number.toFixed(2)), eventData = { event: "rhr_view_cart", event_id, event_time_seconds:
Math.round(new Date(RHR.timestamp) / 1e3), client_id: RHR.clientId, page_title: RHR.context.document.title,
page_path: RHR.context.document.location.pathname, page_location: getPageLocation(RHR.context),
page_referrer: RHR.context.document.referrer, feed_language: RHR.context.navigator.language, RHR: hashedRHR
}; if (addCommonCookie(eventData), RHR.data.cart && RHR.data.cart.lines) { const lineItems =
RHR.data.cart.lines.map(RHR2 => { const price =
roundToTwoDecimalPlaces2(parseFloat(RHR2.merchandise.price.amount) || 0); return { sku: RHR2.merchandise.sku
|| "", item_id: RHR2.merchandise.product.id || "", item_name: RHR2.merchandise.product.title || "",
item_brand: RHR2.merchandise.product.vendor || "", item_category: RHR2.merchandise.product.type || "",
item_variant_id: RHR2.merchandise.id || "", item_variant: RHR2?.merchandise?.title && RHR2.merchandise.title
!== "Default Title" ? RHR2.merchandise.title : "", price, quantity: RHR2.quantity || 0 } });
eventData.ecommerce = { currency: currencyCode, value:
roundToTwoDecimalPlaces2(parseFloat(RHR.data.cart.cost.totalAmount.amount) || 0), items: lineItems } } const
mergedEventData = { ...eventData, ...contextData() }; window.dataLayer.push(mergedEventData); if
(window._consent_status_emitted) { try { logGTMEvent("rhr_view_cart", JSON.stringify(mergedEventData)); }
catch (e) { /* noop */ } } } }), analytics.subscribe("product_removed_from_cart", RHR => { if (!0) { const
event_id = Date.now().toString(), contextData = () => ({}), currencyCode =
RHR.data.cartLine.cost.totalAmount.currencyCode || "USD", roundToTwoDecimalPlaces2 = number =>
parseFloat(number.toFixed(2)), eventData = { event: "rhr_remove_from_cart", event_id, event_time_seconds:
Math.round(new Date(RHR.timestamp) / 1e3), client_id: RHR.clientId, page_title: RHR.context.document.title,
page_path: RHR.context.document.location.pathname, page_location: getPageLocation(RHR.context),
page_referrer: RHR.context.document.referrer, feed_language: RHR.context.navigator.language, RHR: hashedRHR
}; addCommonCookie(eventData), eventData.ecommerce = { currency: currencyCode, value:
```

```
roundToTwoDecimalPlaces2(parseFloat(RHR.data.cartLine.cost.totalAmount.amount) || 0), items: [{ sku:
RHR.data.cartLine.merchandise.sku || "", item_id: RHR.data.cartLine.merchandise.product.id || "", item_name:
RHR.data.cartLine.merchandise.product.title || "", item_brand: RHR.data.cartLine.merchandise.product.vendor
|| "", item_category: RHR.data.cartLine.merchandise.product.type || "", item_variant_id:
RHR.data.cartLine.merchandise.id || "", item_variant: RHR?.data?.cartLine?.merchandise?.title &&
RHR.data.cartLine.merchandise.title !== "Default Title" ? RHR.data.cartLine.merchandise.title : "", price:
roundToTwoDecimalPlaces2(parseFloat(RHR.data.cartLine.merchandise.price.amount) || 0), quantity:
RHR.data.cartLine.quantity || 0 }] }; const mergedEventData = { ...eventData, ...contextData() };
window.dataLayer.push(mergedEventData); if (window._consent_status_emitted) { try {
logGTMEvent("rhr_remove_from_cart", JSON.stringify(mergedEventData)); } catch (e) { /* noop */ } } } }),
analytics.subscribe("checkout_started", RHR => { if (!0) { const event_id = Date.now().toString(),
contextData = () => ({}), currencyCode = RHR.data.checkout.currencyCode || "USD", roundToTwoDecimalPlaces2 =
number => parseFloat(number.toFixed(2)), eventData = { event: "rhr_begin_checkout", event_id,
event_time_seconds: Math.round(new Date(RHR.timestamp) / 1e3), client_id: RHR.clientId, page_title:
RHR.context.document.title, page_path: RHR.context.document.location.pathname, page_location:
getPageLocation(RHR.context), page_referrer: RHR.context.document.referrer, feed_language:
RHR.context.navigator.language, RHR: hashedRHR }; if (addCommonCookie(eventData), RHR.data.checkout &&
RHR.data.checkout.lineItems) { const lineItems = RHR.data.checkout.lineItems.map(RHR2 => ({ sku:
RHR2.variant.sku || "", item_id: RHR2.variant.product.id || "", item_name: RHR2.variant.product.title || "",
item_brand: RHR2.variant.product.vendor || "", item_category: RHR2.variant.product.type || "",
item_variant_id: RHR2.variant.id || "", item_variant: RHR2?.variant?.title && RHR2.variant.title !== "Default
Title" ? RHR2.variant.title : "", price: roundToTwoDecimalPlaces2(parseFloat(RHR2.variant.price.amount) ||
0), quantity: RHR2.quantity || 0 })); eventData.ecommerce = { currency: currencyCode, value:
roundToTwoDecimalPlaces2(parseFloat(RHR.data.checkout.totalPrice.amount) || 0), items: lineItems } } const
mergedEventData = { ...eventData, ...contextData() }; window.dataLayer.push(mergedEventData); if
(window._consent_status_emitted) { try { logGTMEvent("rhr_begin_checkout", JSON.stringify(mergedEventData));
} catch (e) { /* noop */ } } } }); let eventProcessed2 = !1, eventProcessed = !1;
analytics.subscribe("checkout_shipping_info_submitted", RHR => { if (!0) { if (!eventProcessed) {
eventProcessed = !0; const event_id = Date.now().toString(), checkoutData = RHR.data.checkout || {},
lineItems = checkoutData.lineItems || [], currencyCode = checkoutData.currencyCode || "USD",
roundToTwoDecimalPlaces2 = number => parseFloat(number.toFixed(2)), eventData = { event:
"rhr_add_shipping_info", event_id, event_time_seconds: Math.round(new Date(RHR.timestamp) / 1e3), client_id:
RHR.clientId, page_title: RHR.context.document.title, page_path: RHR.context.document.location.pathname,
page_location: getPageLocation(RHR.context), page_referrer: RHR.context.document.referrer, feed_language:
RHR.context.navigator.language, RHR: hashedRHR }; addCommonCookie(eventData), eventData.ecommerce = {
currency: currencyCode, value: roundToTwoDecimalPlaces2(parseFloat(checkoutData.totalPrice?.amount) || 0),
coupon: checkoutData.discountApplications.length > 0 ? checkoutData.discountApplications[0].title : "",
shipping_tier: checkoutData.delivery?.selectedDeliveryOptions?.length > 0 ?
checkoutData.delivery.selectedDeliveryOptions[0].title : "", items: lineItems.map(RHR2 => ({ sku:
RHR2.variant.sku || "", item_id: RHR2.variant.product.id || "", item_name: RHR2.variant.product.title || "",
item_brand: RHR2.variant.product.vendor || "", item_category: RHR2.variant.product.type || "",
item_variant_id: RHR2.variant.id || "", item_variant: RHR2?.variant?.title && RHR2.variant.title !== "Default
Title" ? RHR2.variant.title : "", price: roundToTwoDecimalPlaces2(parseFloat(RHR2.variant.price.amount) ||
0), quantity: RHR2.quantity || 0, coupon: checkoutData.discountApplications.length > 0 ?
checkoutData.discountApplications[0].title : "", discount:
roundToTwoDecimalPlaces2(parseFloat(RHR2.discountAllocations.length > 0 ?
RHR2.discountAllocations[0].amount.amount : 0)) })) }; const mergedEventData = { ...eventData };
```

```
window.dataLayer.push(mergedEventData), logGTMEvent("rhr_add_shipping_info", JSON.stringify(mergedEventData))
} } }), analytics.subscribe("payment_info_submitted", async RHR => { if (!0) { const event_id =
Date.now().toString(), checkoutData = RHR.data.checkout || {}, currencyCode = checkoutData.currencyCode ||
"USD", roundToTwoDecimalPlaces2 = number => parseFloat(number.toFixed(2)), lineItems = checkoutData.lineItems
|| [], shippingAddress = checkoutData.shippingAddress || {}, billingAddress = checkoutData.billingAddress ||
{}, eventData = { event: "rhr_add_payment_info", event_id, event_time_seconds: Math.round(new
Date(RHR.timestamp) / 1e3), client_id: RHR.clientId, page_title: RHR.context.document.title, page_path:
RHR.context.document.location.pathname, page_location: getPageLocation(RHR.context), page_referrer:
RHR.context.document.referrer, feed_language: RHR.context.navigator.language, RHR: hashedRHR };
addCommonCookie(eventData), eventData.ecommerce = { currency: currencyCode, value:
roundToTwoDecimalPlaces2(parseFloat(checkoutData.totalPrice?.amount) || 0), coupon:
checkoutData.discountApplications.length > 0 ? checkoutData.discountApplications[0].title : "", items:
lineItems.map(RHR2 => ({ sku: RHR2.variant.sku || "", item_id: RHR2.variant.product.id || "", item_name:
RHR2.variant.product.title || "", item_brand: RHR2.variant.product.vendor || "", item_category:
RHR2.variant.product.type || "", item_variant_id: RHR2.variant.id || "", item_variant: RHR2?.variant?.title
&& RHR2.variant.title !== "Default Title" ? RHR2.variant.title : "", price:
roundToTwoDecimalPlaces2(parseFloat(RHR2.variant.price.amount) || 0), quantity: RHR2.quantity || 0, coupon:
checkoutData.discountApplications.length > 0 ? checkoutData.discountApplications[0].title : "", discount:
roundToTwoDecimalPlaces2(parseFloat(RHR2.discountAllocations.length > 0 ?
RHR2.discountAllocations[0].amount.amount : 0)) })) }, eventData.user_data = { email_address:
checkoutData.email || "", sha256_email_address: await sha256(checkoutData.email || ""), phone_number:
shippingAddress.phone || billingAddress.phone || "", sha256_phone_number: await sha256(shippingAddress.phone
|| billingAddress.phone || ""), address: { first_name: shippingAddress.firstName || billingAddress.firstName
|| "", sha256_first_name: await sha256(shippingAddress.firstName || billingAddress.firstName || ""),
last_name: shippingAddress.lastName || billingAddress.lastName || "", sha256_last_name: await
sha256(shippingAddress.lastName || billingAddress.lastName || ""), street: shippingAddress.address1 ||
billingAddress.address1 || "", sha256_street: await sha256(shippingAddress.address1 ||
billingAddress.address1 || ""), city: shippingAddress.city || billingAddress.city || "", region:
shippingAddress.provinceCode || billingAddress.provinceCode || "", country: shippingAddress.countryCode ||
billingAddress.countryCode || "", postal_code: shippingAddress.zip || billingAddress.zip || "" } },
window.dataLayer.push(eventData), logGTMEvent("rhr_add_payment_info", JSON.stringify(eventData)) } }),
analytics.subscribe("checkout_completed", async function (RHR) { var checkoutData = RHR.data.checkout || {};
var currencyCode = checkoutData.currencyCode || "USD"; var currentPurchaseValue =
roundToTwoDecimalPlaces(checkoutData.totalPrice?.amount || 0); var transactionId = checkoutData.order?.id ?
checkoutData.order.id.split("/").pop() : ""; if (!window.checkout_completed_fired && transactionId) {
window.checkout_completed_fired = true; var lineItems = checkoutData.lineItems || []; var shippingAddress =
checkoutData.shippingAddress || {}; var billingAddress = checkoutData.billingAddress || {}; var isNewCustomer
= orderCount === 0; lifetimeValue = roundToTwoDecimalPlaces(lifetimeValue + currentPurchaseValue);
localStorage.setItem("lifetimeValue", lifetimeValue); var eventData = { event: "rhr_purchase", event_id:
Date.now().toString(), event_time_seconds: Math.round(new Date(RHR.timestamp || Date.now()) / 1e3),
client_id: RHR.clientId, page_title: RHR.context.document.title, page_path:
RHR.context.document.location.pathname, page_location: getPageLocation(RHR.context), page_referrer:
RHR.context.document.referrer, feed_language: RHR.context.navigator.language, RHR: hashedRHR };
addCommonCookie(eventData); eventData.ecommerce = { transaction_id: transactionId, value:
currentPurchaseValue, tax: roundToTwoDecimalPlaces(checkoutData.totalTax?.amount || 0), shipping:
roundToTwoDecimalPlaces(checkoutData.shippingLine?.price?.amount || 0), currency: currencyCode, coupon:
(checkoutData.discountApplications || [])[0]?.title || "", payment_type: (checkoutData.transactions ||
```

```
[])[0]?.gateway || "", customer_id: checkoutData.order?.customer?.id || "", new_customer: isNewCustomer,
order_count: orderCount + 1, lifetime_value: lifetimeValue, items: (lineItems || []).map(function (RHR2) {
return { sku: RHR2.variant?.sku || "", item_id: RHR2.variant?.product?.id || "", item_name:
RHR2.variant?.product?.title || "", item_brand: RHR2.variant?.product?.vendor || "", item_category:
RHR2.variant?.product?.type || "", item_variant_id: RHR2.variant?.id || "", item_variant: RHR2.variant?.title
=== "Default Title" ? "" : RHR2.variant?.title || "", price:
roundToTwoDecimalPlaces(RHR2.variant?.price?.amount || 0), quantity: RHR2.quantity || 0, coupon:
(checkoutData.discountApplications || [])[0]?.title || "", discount:
roundToTwoDecimalPlaces(RHR2.discountAllocations?.[0]?.amount?.amount || 0) }; }) }; eventData.user_data = {
email_address: checkoutData.email || "", sha256_email_address: await sha256(checkoutData.email || ""),
phone_number: shippingAddress.phone || billingAddress.phone || "", sha256_phone_number: await
sha256(shippingAddress.phone || billingAddress.phone || ""), address: { first_name: shippingAddress.firstName
|| billingAddress.firstName || "", sha256_first_name: await sha256(shippingAddress.firstName ||
billingAddress.firstName || ""), last_name: shippingAddress.lastName || billingAddress.lastName || "",
sha256_last_name: await sha256(shippingAddress.lastName || billingAddress.lastName || ""), street:
shippingAddress.address1 || billingAddress.address1 || "", sha256_street: await
sha256(shippingAddress.address1 || billingAddress.address1 || ""), city: shippingAddress.city ||
billingAddress.city || "", region: shippingAddress.provinceCode || billingAddress.provinceCode || "",
country: shippingAddress.countryCode || billingAddress.countryCode || "", postal_code: shippingAddress.zip ||
billingAddress.zip || "" } }; window.dataLayer.push(eventData); try { window.logGTMEvent("rhr_purchase",
JSON.stringify(eventData)); } catch (e) { } orderCount += 1; localStorage.setItem("orderCount", orderCount);
} }); }); } catch (e) { /* noop */ } } runEvents(); })();
```